



Chapter 8

Project 2.5: Schema and Metadata

It helps to keep the data schema separate from the various applications that share the schema. One way to do this is to have a separate module with class definitions that all of the applications in a suite can share. While this is helpful for a simple project, it can be awkward when sharing data schema more widely. A Python language module is particularly difficult for sharing data outside the Python environment.

This project will define a schema in JSON Schema Notation, first by building `pydantic` class definitions, then by extracting the JSON from the class definition. This will allow you to publish a formal definition of the data being created. The schema can be used by a variety of tools to validate data files and assure that the data is suitable for further analytical use.

The schema is also useful for diagnosing problems with data sources. Validator tools like `jsonschema` can provide detailed error reports that can help identify changes in source data from bug fixes or software updates.

This chapter will cover a number of skills related to data inspection techniques:

- Using the **Pydantic** module for crisp, complete definitions
- Using JSON Schema to create an exportable language-independent definition that anyone can use
- Creating test scenarios to use the formal schema definition

We'll start by looking at the reasons why a formal schema is helpful.

8.1 Description

Data validation is a common requirement when moving data between applications. It is extremely helpful to have a clear definition of what constitutes valid data. It helps even more when the definition exists outside a particular programming language or platform.

We can use the JSON Schema (<https://json-schema.org>) to define a schema that applies to the intermediate documents created by the acquisition process. Using JSON Schema enables the confident and reliable use of the JSON data format.

The JSON Schema definition can be shared and reused within separate Python projects and with non-Python environments, as well. It allows us to build data quality checks into the acquisition pipeline to positively affirm the data really fit the requirements for analysis and processing.

Additional metadata provided with a schema often includes the provenance of the data and details on how attribute values are derived. This isn't a formal part of a JSON Schema, but we can add some details to the JSON Schema document that includes provenance and processing descriptions.

The subsequent data cleaning projects should validate the input documents using a source schema. Starting with [***Chapter 9, Project 3.1: Data Cleaning Base Application***](#), the applications should validate their output using the target analytic schema. It can seem silly to have an application both create sample records and also validate those records against a schema. What's important is the schema will be shared, and evolve with the needs of consumers of the data. The data acquisition and cleaning operations, on the other hand, evolve with

the data sources. It is all too common for an ad hoc solution to a data problem to seem good but create invalid data.

It rarely creates new problems to validate inputs as well as outputs against a visible, agreed-upon schema. There will be some overhead to the validation operation, but much of the processing cost is dominated by the time to perform input and output, not data validation.

Looking forward to [*Chapter 12, Project 3.8: Integrated Data Acquisition Web Service*](#), we'll see additional uses for a formally-defined schema. We'll also uncover a small problem with using JSON Schema to describe ND JSON documents. For now, we'll focus on the need to use JSON Schema to describe data.

We'll start by adding some modules to make it easier to create JSON Schema documents.

8.2 Approach

First, we'll need some additional modules. The `jsonschema` module defines a validator that can be used to confirm a document matches the defined schema.

Additionally, the **Pydantic** module provides a way to create class definitions that can emit JSON Schema definitions, saving us from having to create the schema manually. In most cases, manual schema creation is not terribly difficult. For some cases, though, the schema and the validation rules might be challenging to write directly, and having Python class definitions available can simplify the process.

This needs to be added to the `requirements-dev.txt` file so other developers know to install it.

When using **conda** to manage virtual environments, the command might look like the following:

```
% conda install jsonschema pydantic
```

When using other tools to manage virtual environments, the command might look like the following:

```
% python -m pip install jupyterlab
```

The JSON Schema package requires some supplemental type stubs. These are used by the **mypy** tool to confirm the application is using types consistently. Use the following command to add stubs:

```
% mypy --install-types
```

Additionally, the **pydantic** package includes a **mypy** plug-in that will extend the type-checking capabilities of **mypy**. This will spot more nuanced potential problems with classes defined using **pydantic**.

To enable the plugin, add **pydantic.mypy** to the list of plugins in the **mypy** configuration file, **mypy.ini**. The **mypy.ini** file should look like this:

```
[mypy]
plugins = pydantic.mypy
```

(This file goes in the root of the project directory.)

This plugin is part of the **pydantic** download, and is compatible with **mypy** versions starting with 0.910.

With these two packages, we can define classes with details that can be used to create JSON Schema files. Once we have a JSON Schema file, we can use the schema definition to confirm that sample data is valid.

For more information on **Pydantic**, see <https://docs.pydantic.dev>.

The core concept is to use **Pydantic** to define dataclasses with detailed field definitions. These definitions can be used for data validation in Python. The definition can also be used to emit a JSON Schema document to share with other projects.

The schema definitions are also useful for defining an OpenAPI specification. In [*Chapter 12, Project 3.8: Integrated Data Acquisition Web Service*](#), we'll turn to creating a web service that provides data. The OpenAPI specification for this service will include the schema definitions from this project.

The use of **Pydantic** isn't required. It is, however, very convenient for creating a schema that can be described via JSON Schema. It saves a great deal of fussing with details in JSON syntax.

We'll start with using **Pydantic** to create a useful data model module. This will extend the data models built for projects in earlier chapters.

8.2.1 Define Pydantic classes and emit the JSON Schema

We'll start with two profound modifications to the data model definitions used in earlier chapters. One change is to switch from the `dataclasses` module to the `pydantic.dataclasses` module. Doing this creates the need to explicitly use `dataclasses.field` for individual field definitions. This is generally a small change to an `import` statement to use `from pydantic.dataclasses import dataclass`. The `dataclasses.field()` function will need some changes, also, to add additional details used by **pydantic**. The changes should be completely transparent to the existing application; all tests will pass after these changes.

The second change is to add some important metadata to the classes. Where the `dataclasses.field(...)` definition is used, the

`metadata={}` attribute can be added to include a dictionary with JSON Schema attributes like the description, title, examples, valid ranges of values, etc. For other fields, the `pydantic.Field()` function must be used to provide a title, description, and other constraints on the field. This will generate a great deal of metadata for us.

See <https://docs.pydantic.dev/usage/schema/#field-customization> for the wide variety of field definition details available.

```
from pydantic import Field
from pydantic.dataclasses import dataclass

@dataclass
class SeriesSample:
    """
    An individual sample value.
    """
    x: float = Field(title="The x attribute", ge=0.0)
    y: float = Field(title="The y attribute", ge=0.0)

@dataclass
class Series:
    """
    A named series with a collection of values.
    """
    name: str = Field(title="Series name")
    samples: list[SeriesSample] = Field(title="Sequence of samples
        in this series")
```

We've provided several additional details in this model definition module. The details include:

- Docstrings on each class. These will become descriptions in the JSON Schema.
- Fields for each attribute. These, too, become descriptions in the JSON Schema.
- For the `x` and `y` attributes of the `SeriesSample` class definition, we added a `ge` value. This is a range specification, requiring the

values to be greater than or equal to zero.

We've also made extremely profound changes to the model: we've moved from the source data description — which was a number of `str` values — to the target data description, using `float` values.

What's central here is that we have two variations on each model:

- **Acquisition:** This is the data as we find it "in the wild." In the examples in this book, some variations of source data are text-only, forcing us to use `str` as a common type. Some data sources will have data in more useful Python objects, permitting types other than `str`.
- **Analysis:** This is the data used for further analysis. These data sets can use native Python objects. For the most part, we'll focus on objects that are easily serialized to JSON. The exception will be date-time values, which don't readily serialize to JSON, but require some additional conversion from a standard ISO text format.

The class examples shown above do not *replace* the `model` module in our applications. They form a second model of more useful data. The recommended approach is to change the initial acquisition model's module name from `model` to `acquisition_model` (or perhaps the shorter `source_model`). This property describes the model with mostly string values as the source. This second model is the `analysis_model`.

The results of the initial investigation into the data can provide narrower and more strict constraints for the analysis model class definitions. See [*Chapter 7, Data Inspection Features*](#) for a number of inspections that can help to reveal expected minima and maxima for attribute values.

The **Pydantic** library comes with a large number of customized data types that can be used to describe data values. See

<https://docs.pydantic.dev/usage/types/> for documentation. Using the `pydantic` types can be simpler than defining an attribute as a string, and trying to create a regular expression for valid values.

Note that validation of source values isn't central to **Pydantic**. When Python objects are provided, it's entirely possible for the **Pydantic** module to perform a successful data conversion where we might have hoped for an exception to be raised. A concrete example is providing a Python `float` object to a field that requires an `int` value. The `float` object will be converted; an exception will *not* be raised. If this kind of very strict validation of Python objects is required, some additional programming is needed.

In the next section, we'll create a JSON Schema definition of our model. We can either export the definition from the class definition, or we can craft the JSON manually.

8.2.2 Define expected data domains in JSON Schema notation

Once we have the class definition, we can then export a schema that describes the class. Note that the **Pydantic** dataclass is a wrapper around an underlying `pydantic.BaseModel` subclass definition.

We can create a JSON Schema document by adding the following lines to the bottom of the module:

```
from pydantic import schema_of
import json

if __name__ == "__main__":
    schema = schema_of(Series)
    print(json.dumps(schema, indent=2))
```

These lines turn the data definition module into a script that writes the JSON Schema definition to the standard output file.

The `schema_of()` function will extract a schema from the dataclass created in the previous section. (See [***Define Pydantic classes and emit the JSON Schema***](#).) The underlying `pydantic.BaseModel` subclass also has a `schema()` method that will transform the class definition into a richly-detailed JSON Schema definition. When working with **pydantic** dataclasses, the `pydantic.BaseModel` isn't directly available, and the `schema_of()` function must be used.

When executing the terminal command

```
python src/analysis_model.py
```

, the schema is displayed.

The output begins as follows:

```
{
  "title": "Series",
  "description": "A named series with a collection of values.",
  "type": "object",
  "properties": {
    "name": {
      "title": "Series name",
      "type": "string"
    },
    "samples": {
      "title": "Sequence of samples in this series",
      "type": "array",
      "items": {
        "$ref": "#/definitions/SeriesSample"
      }
    }
  },
  "required": [
    "name",
    "samples"
  ],
  ...
}
```

We can see that the title matches the class name. The description matches the docstring. The collection of properties matches the attributes' names in the class. Each of the property definitions provides the type information from the dataclass.

The `$ref` item is a reference to another definition provided later in the JSON Schema. This use of references makes sure the other class definition is separately visible, and is available to support this schema definition.

A very complex model may have a number of definitions that are shared in multiple places. This `$ref` technique normalizes the structure so only a single definition is provided. Multiple references to the single definition assure proper reuse of the class definition.

The JSON structure may look unusual at first glance, but it's not frighteningly complex. Reviewing <https://json-schema.org> will provide information on how best to create JSON Schema definitions without using the **Pydantic** module.

8.2.3 Use JSON Schema to validate intermediate files

Once we have a JSON Schema definition, we can provide it to other stakeholders to be sure they understand the data required or the data provided. We can also use the JSON Schema to create a validator that can examine a JSON document and determine if the document really does match the schema.

We can do this with a `pydantic` class definition. There's a `parse_obj()` method that will examine a dictionary to create an instance of the given `pydantic` class could be built. The `parse_raw()` method can parse a string or bytes object to create an instance of the given class.

We can also do this with the `jjsonschema` module. We'll look at this as an alternative to `pydantic` to show how sharing the JSON Schema allows other applications to work with a formal definition of the analysis model.

First, we need to create a validator from the schema. We can dump the JSON into a file and then load the JSON back from the file. We can also save a step by creating a validator directly from the **Pydantic**-created JSON Schema. Here's the short version:

```
from pydantic import schema_of
from jjsonschema.validators import Draft202012Validator
from analysis_model import *

schema = schema_of(SeriesSample)
validator = Draft202012Validator(schema)
```

This creates a validator using the latest version of JSON Schema, the 2020 draft. (The project is on track to become a standard, and has gone through a number of drafts as it matures.)

Here's how we might write a function to scan a file to be sure the NDJSON documents all properly fit the defined schema:

```
def validate_ndjson_file(
    validator: Draft202012Validator,
    source_file: TextIO
) -> Counter[str]:
    counts: Counter[str] = Counter()
    for row in source_file:
        document = json.loads(row)
        if not validator.is_valid(document):
            errors = list(validator.iter_errors(document))
            print(document, errors)
            counts['faulty'] += 1
    else:
```

```
counts['good'] += 1
return counts
```

This function will read each NDJSON document from the given source file. It will use the given validator to see if the document has problems or is otherwise valid. For faulty documents, it will print the document and the entire list of validation errors.

This kind of function can be embedded into a separate script to check files.

We can, similarly, create the schema for the source model, and use JSON Schema (or **Pydantic**) to validate source files before attempting to process them.

We'll turn to the more complete validation and cleaning solution in [**Chapter 9, Project 3.1: Data Cleaning Base Application**](#). This project is one of the foundational components of the more complete solution.

We'll look at the deliverables for this project in the next section.

8.3 Deliverables

This project has the following deliverables:

- A `requirements.txt` file that identifies the tools used, usually `pydantic==1.10.2` and `jjsonschema==4.16.0`.
- Documentation in the `docs` folder.
- The JSON-format files with the source and analysis schemas. A separate `schema` directory is the suggested location for these files.
- An acceptance test for the schemas.

We'll look at the schema acceptance test in some detail. Then we'll look at using schema to extend other acceptance tests.

8.3.1 Schema acceptance tests

To know if the schema is useful, it is essential to have acceptance test cases. As new sources of data are integrated into an application, and old sources of data mutate through ordinary bug fixes and upgrades, files will change. The new files will often cause problems, and the root cause of the problem will be the unexpected file format change.

Once a file format change is identified, the smallest relevant example needs to be transformed into an acceptance test. The test will — of course — fail. Now, the data acquisition pipeline can be fixed knowing there is a precise definition of done.

To start with, the acceptance test suite should have an example file that's valid and an example file that's invalid.

As we noted in [***Chapter 4, Data Acquisition Features: Web APIs and Scraping***](#), we can provide a large block of text as part of a Gherkin scenario. We can consider something like the following scenario:

```
Scenario: Valid file is recognized.  
  Given a file "example_1.ndjson" with the following content  
    """  
    {"x": 1.2, "y": 3.4}  
    {"x": 5.6, "y": 7.8}  
    """  
  
  When the schema validation tool is run with the analysis schema  
  Then the output shows 2 good records  
  And the output shows 0 faulty records
```

This allows us to provide the contents for an NDJSON file. The HTML extract command is quite long. The content is available as the `context.text` parameter of the step definition function. See [***Acceptance tests***](#) for more examples of how to write the step definitions to create a temporary file to be used for this test case.

Scenarios for faulty records are also essential, of course. It's important to be sure the schema definition will reject invalid data.

8.3.2 Extended acceptance testing

In *Chapters 3, 4, and 5*, we wrote acceptance tests that — generally — looked at log summaries of the application’s activity to be sure it properly acquired source data. We did not write acceptance tests that specifically looked at the data.

Testing with a schema definition permits a complete analysis of each and every field and record in a file. The completeness of this check is of tremendous value.

This means that we can add some additional Then steps to existing scenarios. They might look like the following:

```
# Given (shown earlier)...
# When (shown earlier)...
Then the log has an INFO line with "header: ['x', 'y']"
And log has INFO line with "Series_1 count: 11"
And log has INFO line with "Series_2 count: 11"
And log has INFO line with "Series_3 count: 11"
And log has INFO line with "Series_4 count: 11"
And the output directory files are valid
    using the "schema/Anscombe_Source.json" schema
```

The additional “Then the output directory files are valid...” line requires a step definition that must do the following things:

1. Load the named JSON Schema file and build a `Validator`.
2. Use the `Validator` object to examine each line of the ND JSON file to be sure they’re valid.

This use of the schema as part of the acceptance test suite will parallel the way data suppliers and data consumers can use the schema to assure the data files are valid.

It’s important to note the schema definition given earlier in this chapter (in *[Define Pydantic classes and emit the JSON Schema](#)*) was the

output from a future project's data cleaning step. The schema shown in that example is not the output from the previous data acquisition applications.

To validate the output from data acquisition, you will need to use the model for the various data acquisition projects in *Chapters 3, 4, and 5*. This will be **very** similar to the example shown earlier in this chapter. While similar, it will differ in a profound way: it will use `str` instead of `float` for the series sample attribute values.

8.4 Summary

This chapter's projects have shown examples of the following features of a data acquisition application:

- Using the Pydantic module for crisp, complete definitions
- Using JSON Schema to create an exportable language-independent definition that anyone can use
- Creating test scenarios to use the formal schema definition

Having formalized schema definitions permits recording additional details about the data processing applications and the transformations applied to the data.

The docstrings for the class definitions become the descriptions in the schema. This permits writing details on data provenance and transformation that are exposed to all users of the data.

The JSON Schema standard permits recording examples of values. The **Pydantic** package has ways to include this metadata in field definitions, and class configuration objects. This can be helpful when explaining odd or unusual data encodings.

Further, for text fields, JSONSchema permits including a format attribute that can provide a regular expression used to validate the text.

The **Pydantic** package has first-class support for this additional validation of text fields.

We'll return to the details of data validation in [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#) and [*Chapter 10, Data Cleaning Features*](#). In those chapters, we'll delve more deeply into the various **Pydantic** validation features.

8.5 Extras

Here are some ideas for you to add to this project.

8.5.1 Revise all previous chapter models to use Pydantic

The previous chapters used `dataclass` definitions from the `dataclasses` module. These can be shifted to use the `pydantic.dataclasses` module. This should have minimal impact on the previous projects.

We can also shift all of the previous acceptance test suites to use a formal schema definition for the source data.

8.5.2 Use the ORM layer

For SQL extracts, an ORM can be helpful. The `pydantic` module lets an application create Python objects from intermediate ORM objects. This two-layer processing seems complex but permits detailed validation in the **Pydantic** objects that aren't handled by the database.

For example, a database may have a numeric column without any range provided. A **Pydantic** class definition can provide a field definition with `ge` and `le` attributes to define a range. Further, **Pydantic** permits the definition of a unique data type with unique validation rules that can be applied to database extract values.

First, see <https://docs.sqlalchemy.org/en/20/orm/> for information on the SQLAlchemy ORM layer. This provides a class definition from which SQL statements like `CREATE TABLE`, `SELECT`, and `INSERT` can be derived.

Then, see the <https://docs.pydantic.dev/usage/models/#orm-mode-aka-arbitrary-class-instances> "ORM Mode (aka Arbitrary Class Instances)" section of the **Pydantic** documentation for ways to map a more useful class to the intermediate ORM class.

For legacy data in a quirky, poorly-designed database, this can become a problem. For databases designed from the beginning with an ORM layer, on the other hand, this can be a simplification to the SQL.